

Enterprise Application Development – AI

1. Created Confluence page using vibe coding

- Created .md files and started uploading pages into the confluence .
- Provide required confluence api keys ,space information ,api keys created with the scopes
- Calling confluence api for uploading the pages

Prompt below used to conver .md files to documents -> it created .html files under ouputs/a.html for each .md file . Once it is generated it will upload all pages to confluence via OAuth.

You are a banking documentation assistant.

Convert the following Markdown into Confluence-ready documentation.

Enhance structure if needed:

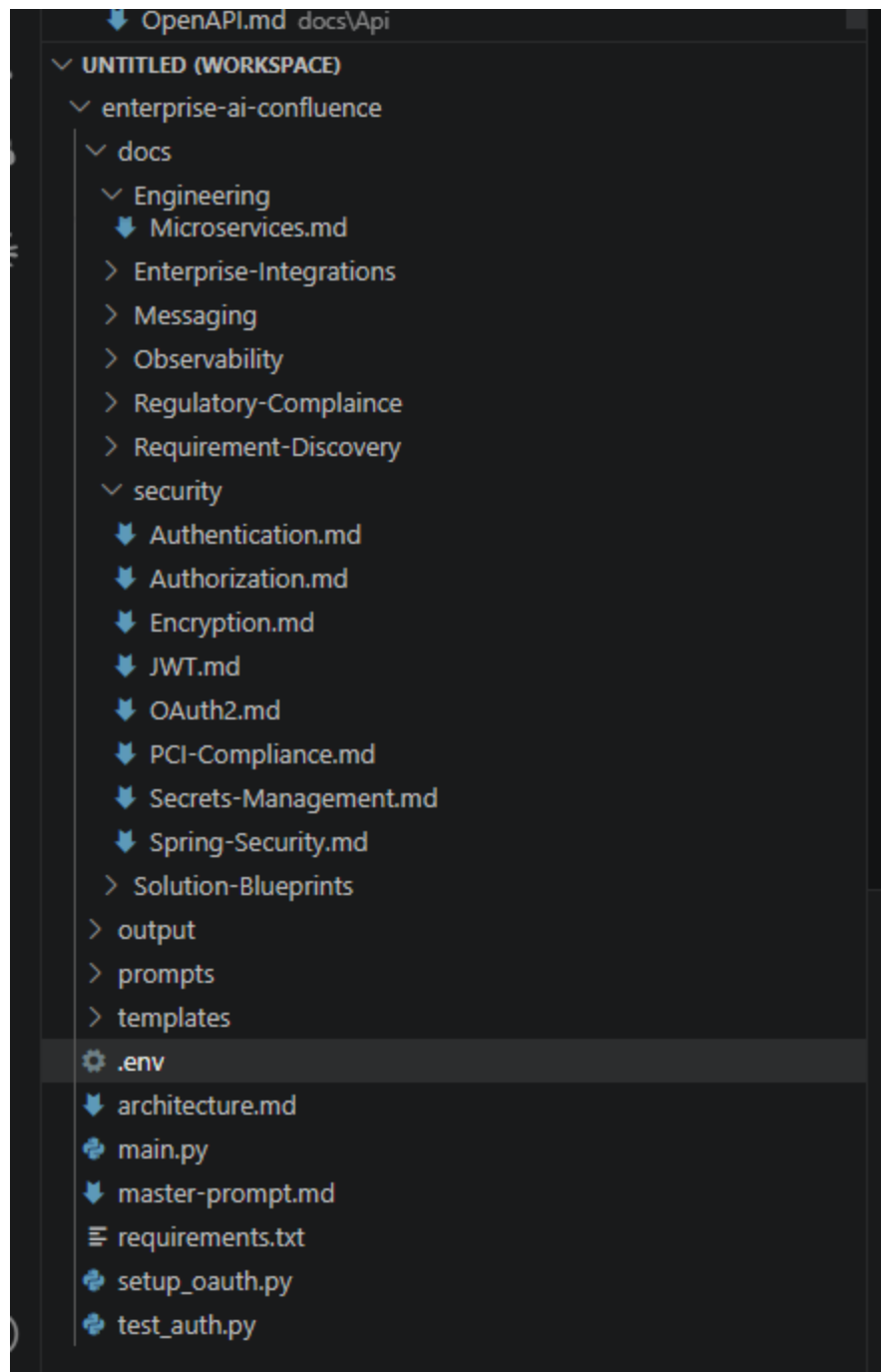
- Add missing sections (Overview, Architecture, APIs, Business Rules)
 - Keep banking domain accuracy
 - Maintain professional enterprise tone
- Output must be Confluence storage format compatible HTML

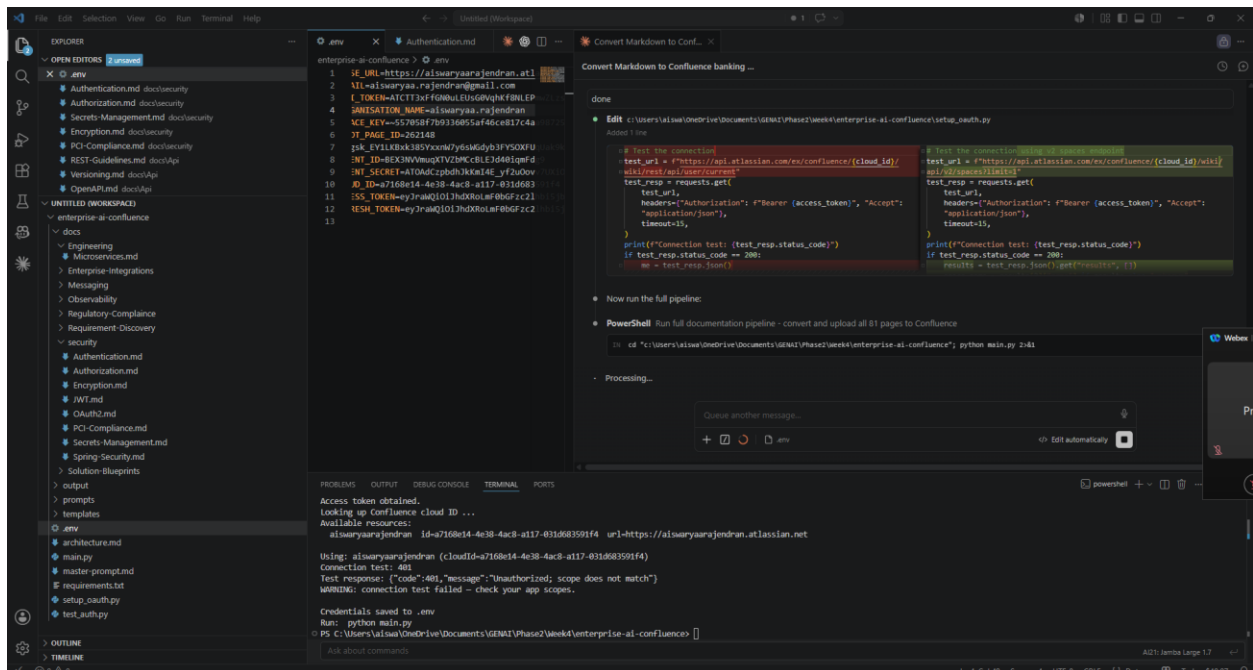
Markdown content:

{md_file_content}

Markdown content:

{md_file_content}





Make sure we have right scope added to api key and enable OAuth app

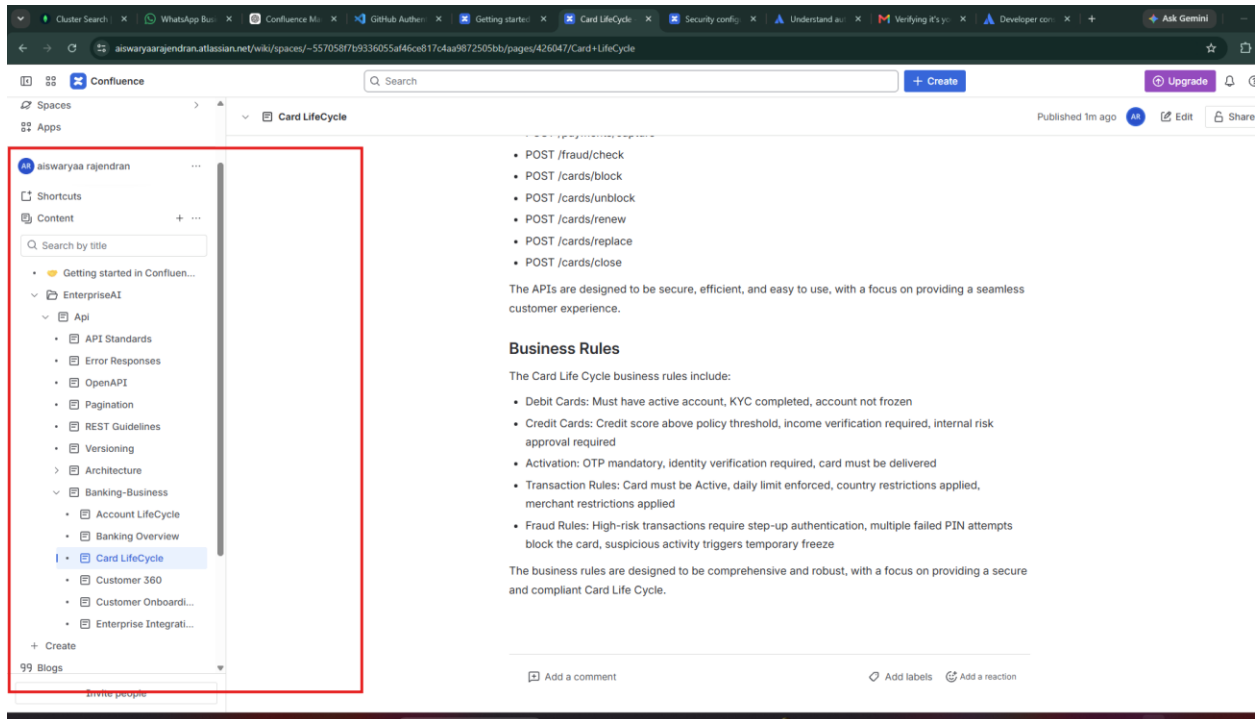
Go to your OAuth app in Atlassian Developer Console

1. Open: <https://developer.atlassian.com/console/myapps/>
2. Click your app "**Confluence Uploader**"
3. Go to "**Permissions**" → "**Confluence API**" → "**Edit Scopes**"
4. Add / enable these **granular scopes**:
5. read:page:confluence
6. write:page:confluence
7. read:space:confluence

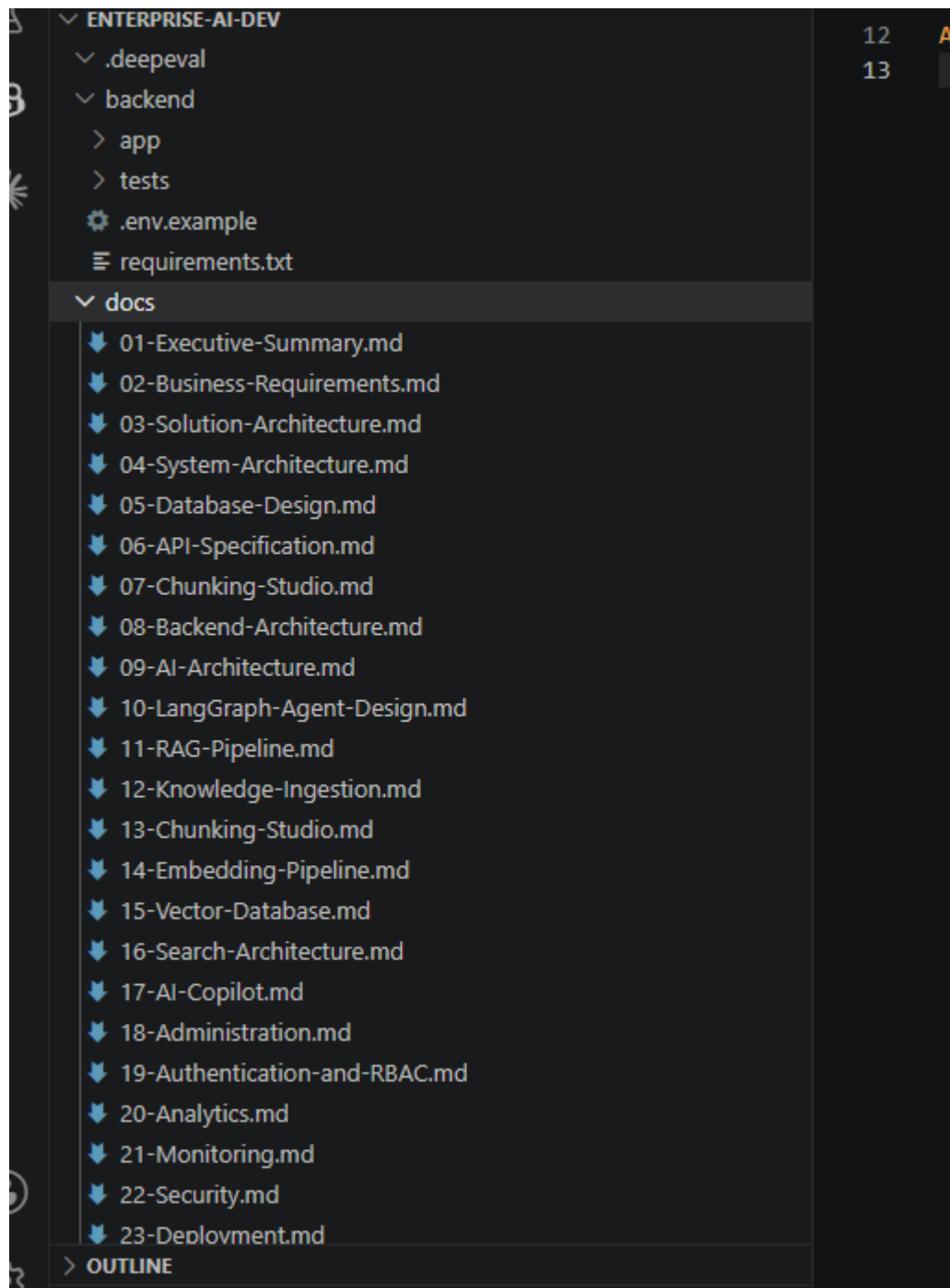
(Keep offline_access too)

8. Save

Confluence page verification:



2. Created .md files placed under project/docs/



3. Create Backend front end development by using the generated prompt using chatgpt

proceed

Frontend Implementation Complete ✓

Production build: ✓ built in 5.04s — 0 TypeScript errors

Files Created (frontend/)

```
src/
├─ vite-env.d.ts      # Vite env type declarations
├─ main.tsx           # React entry point
├─ App.tsx            # Router + QueryClient + AuthProvider
├─ index.css          # Tailwind + custom prose styles
├─ types/index.ts     # All TypeScript interfaces
├─ services/
│   └─ api.ts         # Axios + JWT interceptor + 401 handler
│   └─ auth.service.ts
│   └─ document.service.ts
│   └─ search.service.ts
│   └─ rag.service.ts
│   └─ analytics.service.ts
├─ contexts/AuthContext.tsx # Auth state + login/logout
├─ routes/ProtectedRoute.tsx
└─
```

ctrl esc to focus or unfocus Claude

```
├─ ChatPage.tsx      # Full RAG chat with conversation sidebar + sources
├─ AnalyticsPage.tsx # Charts + top queries + audit log
└─ AdminPage.tsx     # Config, Users, Health tabs
```

How to Run

Backend:

```
cd backend
pip install -r requirements.txt
cp .env.example .env # fill in MONGODB_URI, GROQ_API_KEY
python -m scripts.seed_admin # create first admin user
uvicorn app.main:app --reload
```

Frontend:

```
cd frontend
npm run dev # starts at http://localhost:5173
```

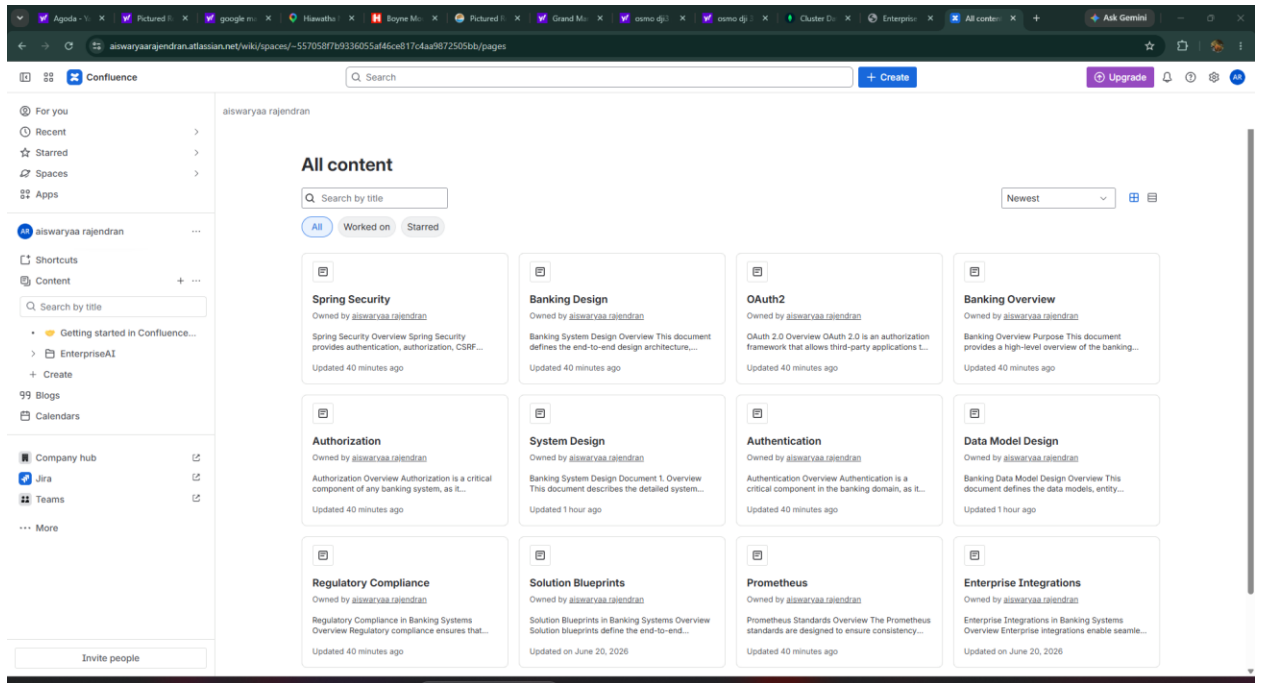
Default admin credentials (after seed): admin@enterprise.ai / Admin@123456

ctrl esc to focus or unfocus Claude

+ | .env

</> Edit automatically

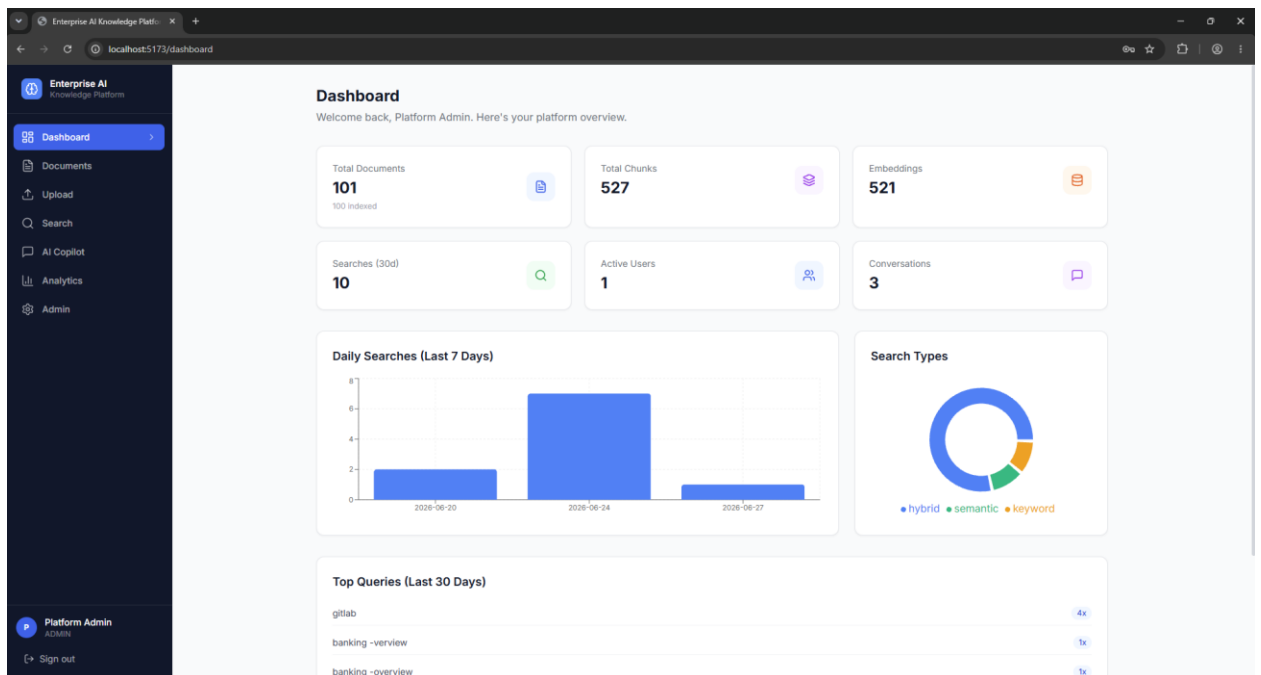
Confluence Page :

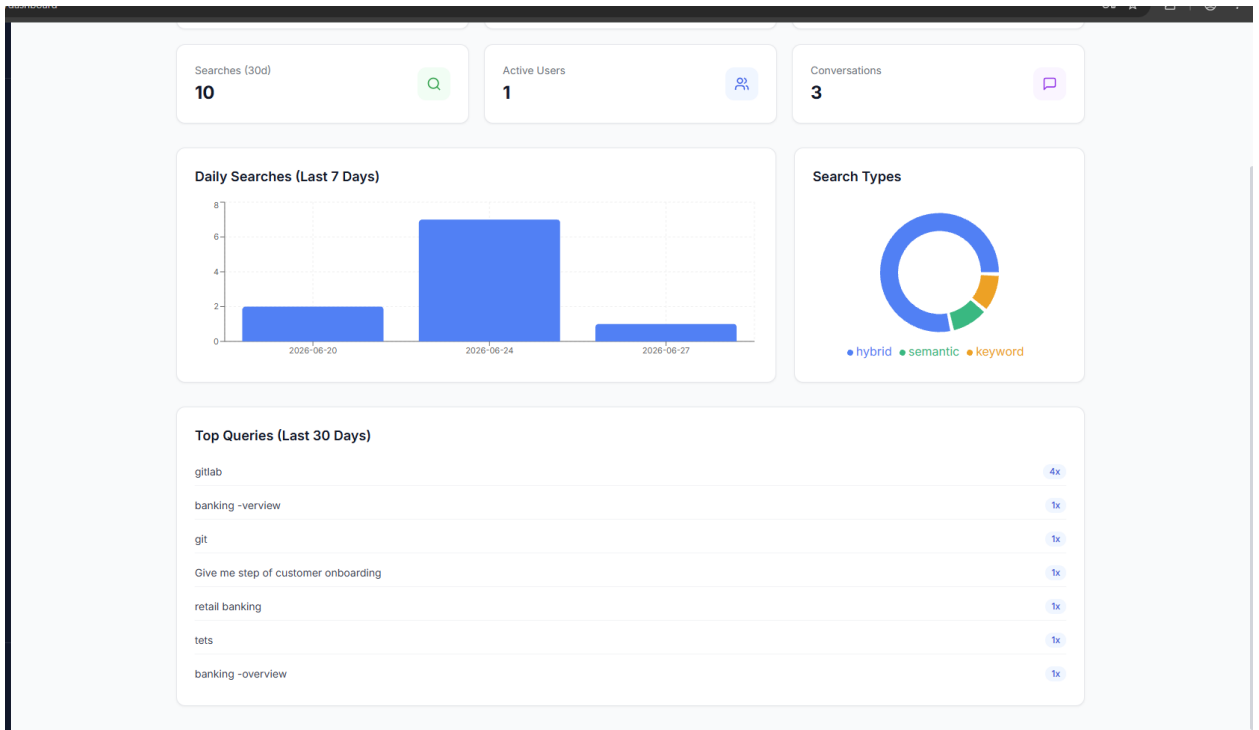


Start backend and front end

admin@enterprise.ai / Admin@123456 – After login to application

Dashboard : Has details of documents , searches ,users details





Clicks on documents -> search result click preview icon – shows up

Enterprise AI Knowledge Platform

Documents: 101 total documents

Search by name...

NAME	TYPE	SIZE	CHUNKS	STATUS
System Design	CONFLUENCE	3.9 KB	6	INDEXED
Banking-Design	CONFLUENCE	70 B	1	INDEXED
Authentication	CONFLUENCE	2.3 KB	4	INDEXED
Alerting	CONFLUENCE	2 KB	4	INDEXED
Event Schema	CONFLUENCE	1.4 KB	3	INDEXED
Account Service	CONFLUENCE	1.6 KB	3	INDEXED
Banking-Domain	CONFLUENCE	70 B	1	INDEXED
Retail Banking	CONFLUENCE	6.4 KB	9	INDEXED
Service Mesh	CONFLUENCE	4.8 KB	7	INDEXED
Event Driven	CONFLUENCE	5.8 KB	9	INDEXED

System Design
Type: CONFLUENCE Version: 1 Uploaded: 6/24/2026 Tags: test

Chunk 1: 3 tokens

Chunk 2: 125 tokens

1. Overview
This document describes the detailed system design for the Enterprise Banking Platform, covering microservice architecture, data flows, integration points, and deployment topology.

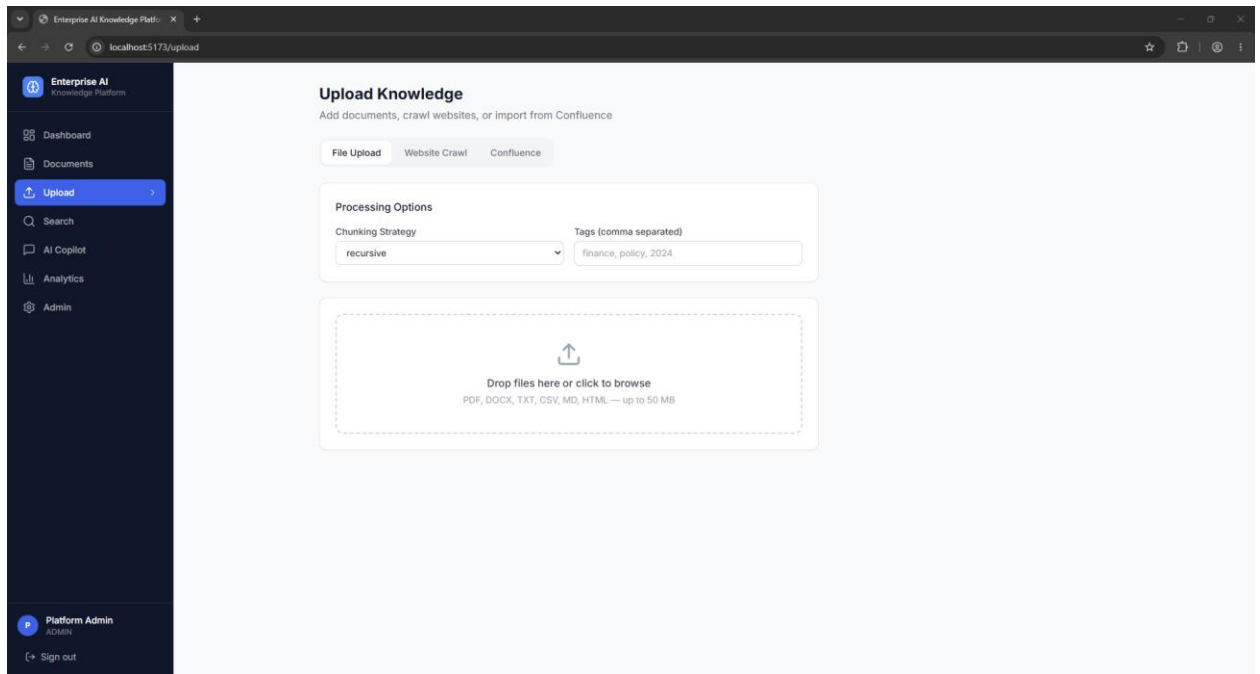
2. System Architecture
2.1 High-Level Design
The high-level design of the system consists of the following layers:
Client Layer: Web Browser, Mobile App, Partner API
API Gateway Layer: Rate Limiting, Auth Filter, SSL Termination, Routing
Service Layer: Auth Service, Account Service, Payment Service, Fraud Service
Message Bus (Kafka): account.events, payment.events, fraud.alerts, audit.logs
Data Layer: MongoDB, PostgreSQL, Redis Cache, Elasticsearch, VectorDB

3. Microservice Design
3.1 Auth Service
The Auth Service is responsible for:
JWT generation and validation
OAuth2 flows
User session management
Database: PostgreSQL (users, roles, permissions)
Cache: Redis (active sessions, refresh tokens)
Endpoints: /auth/login, /auth/refresh, /auth/logout, /auth/validate

Chunk 3: 109 tokens

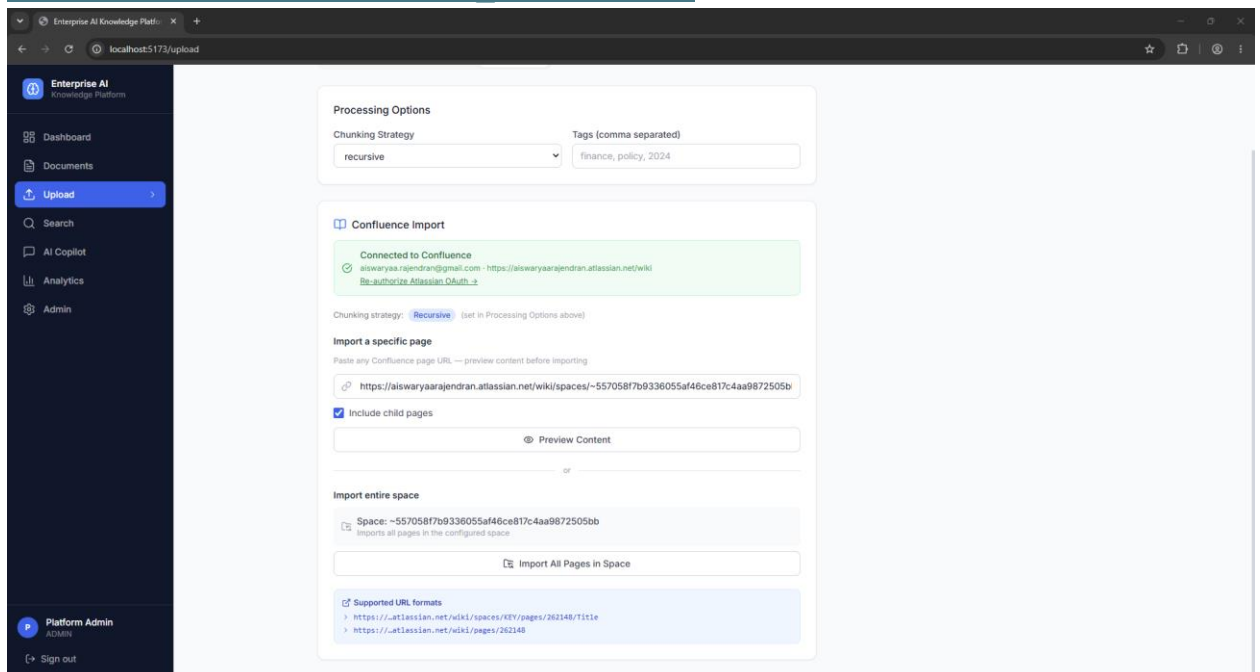
User session management
Database: PostgreSQL (users, roles, permissions)
Cache: Redis (active sessions, refresh tokens)

Upload :

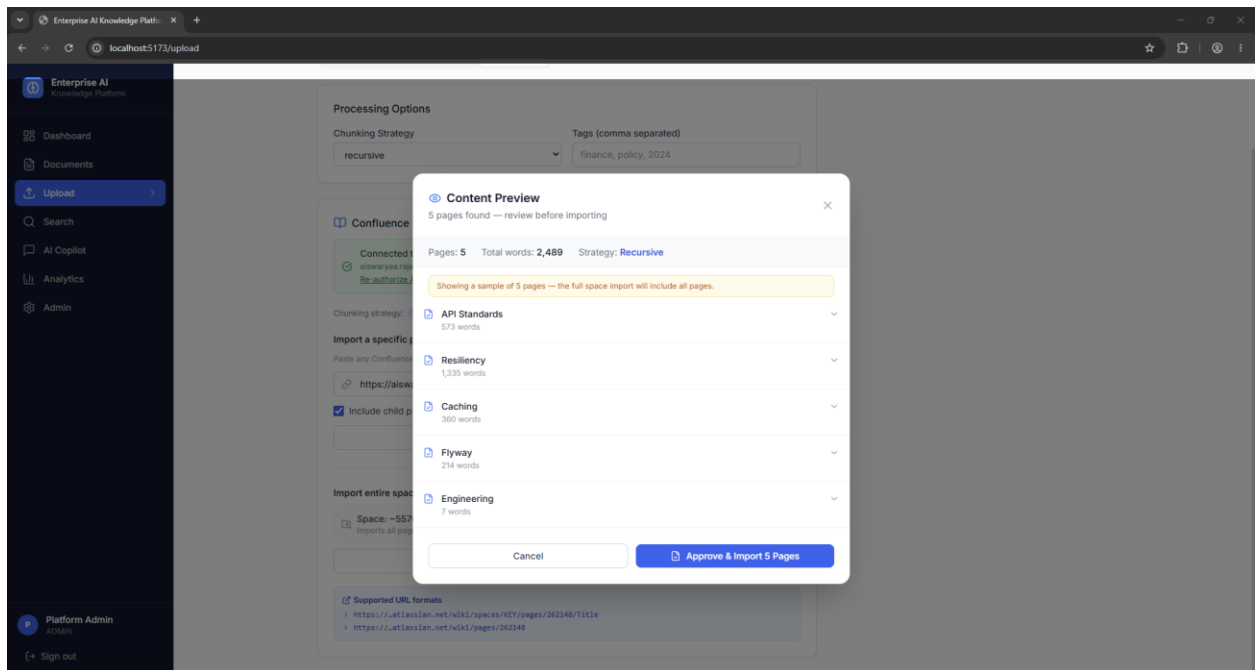


Clicks on Confluence and provide page information -> it will start creating chunks and embedding store it into db.

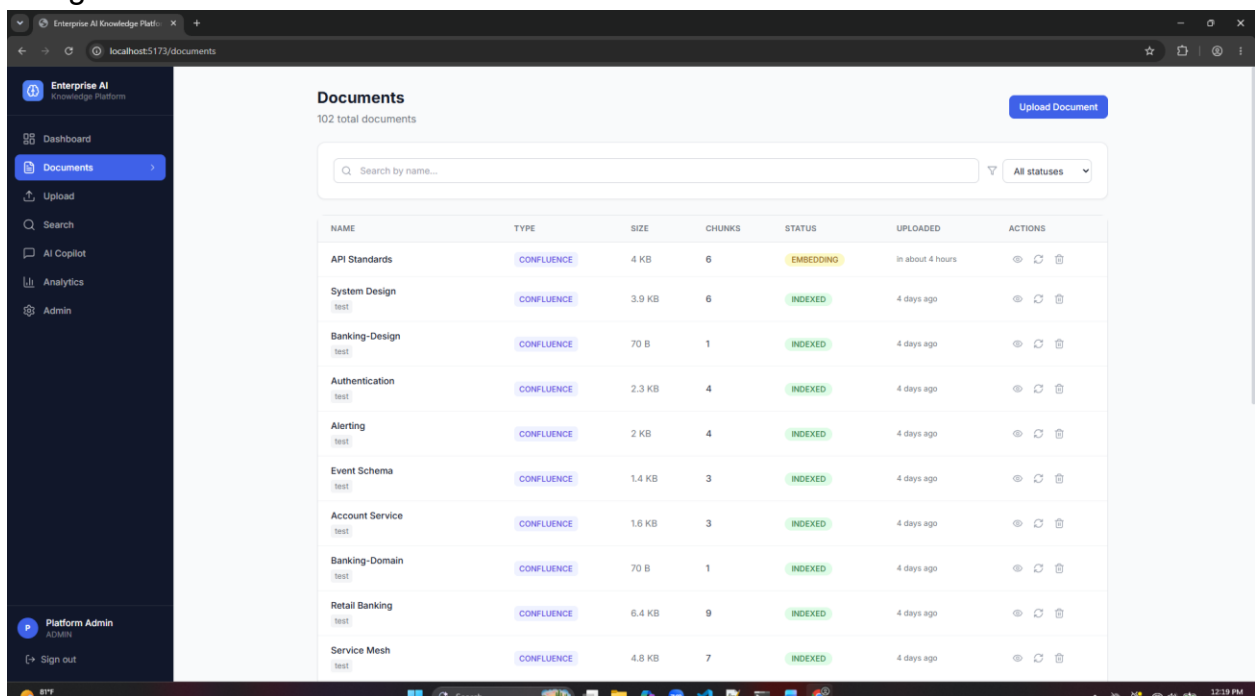
https://aiswaryaarajendran.atlassian.net/wiki/spaces/~557058f7b9336055af46ce817c4aa9872505bb/overview?atl_f=PAGETREE

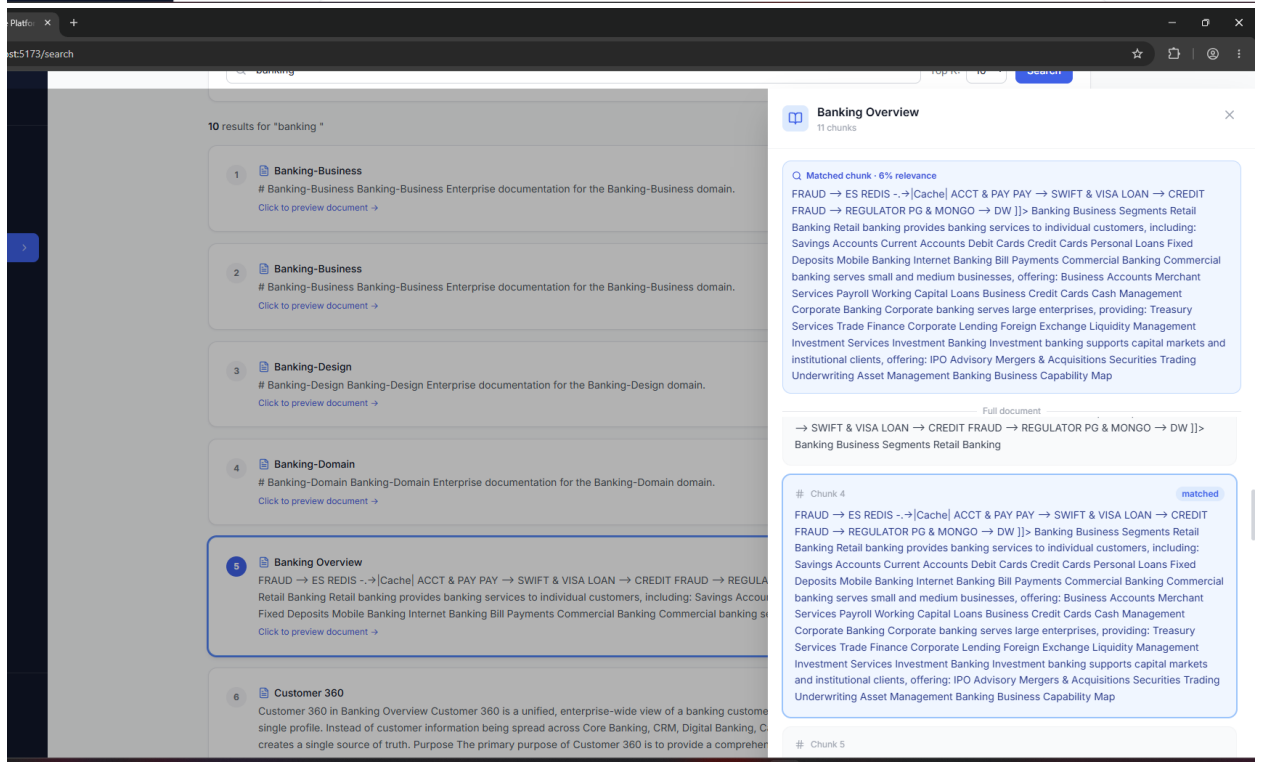
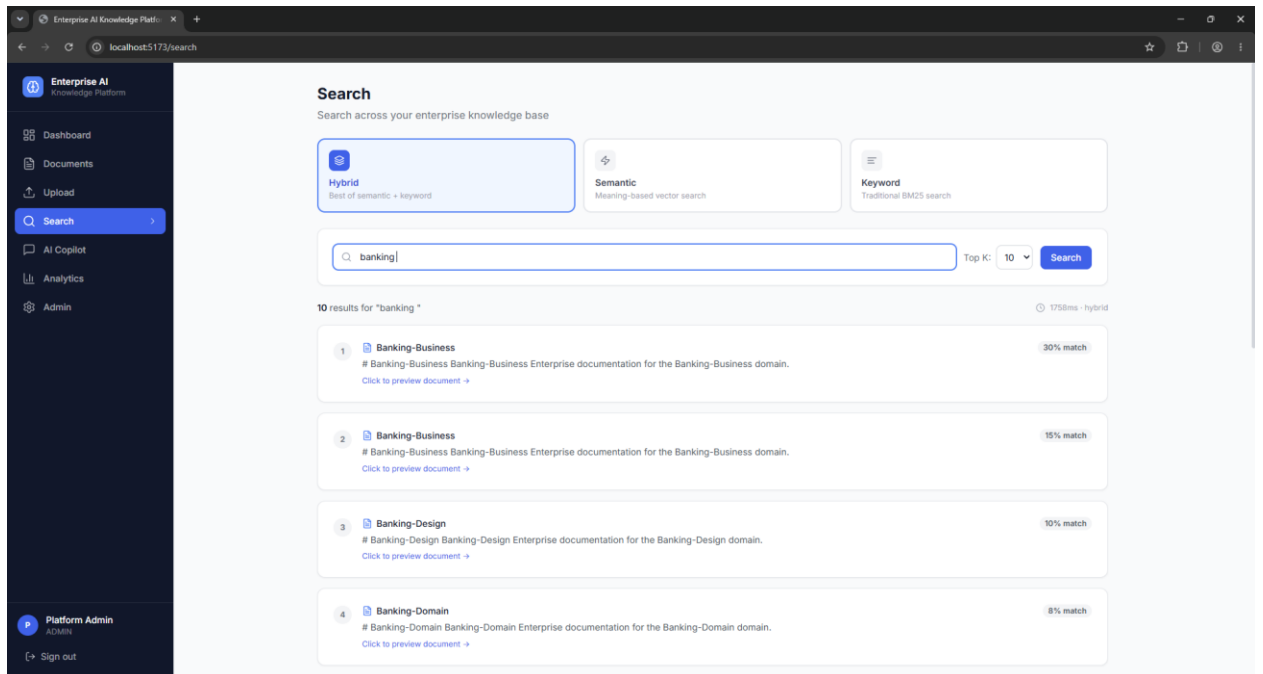


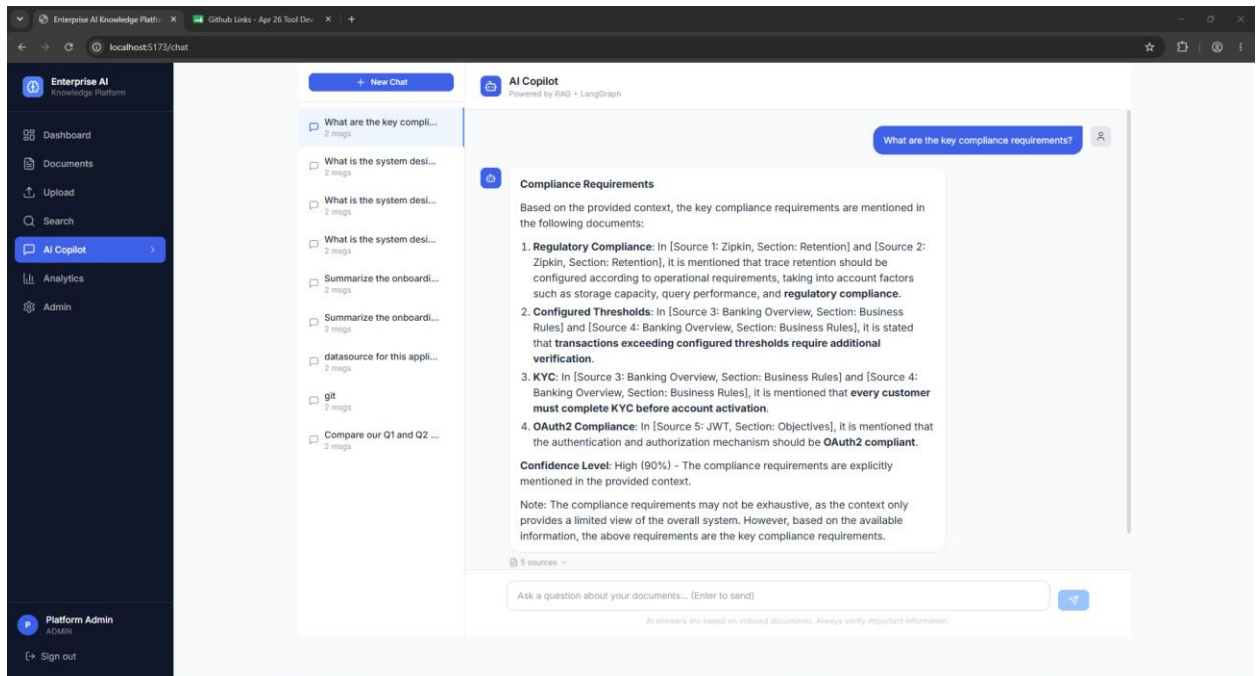
Clicks on Preview content opens up the list of pages fetched from confluence



If user clicks approve it will approve and upload documents with embedding in mongo db .

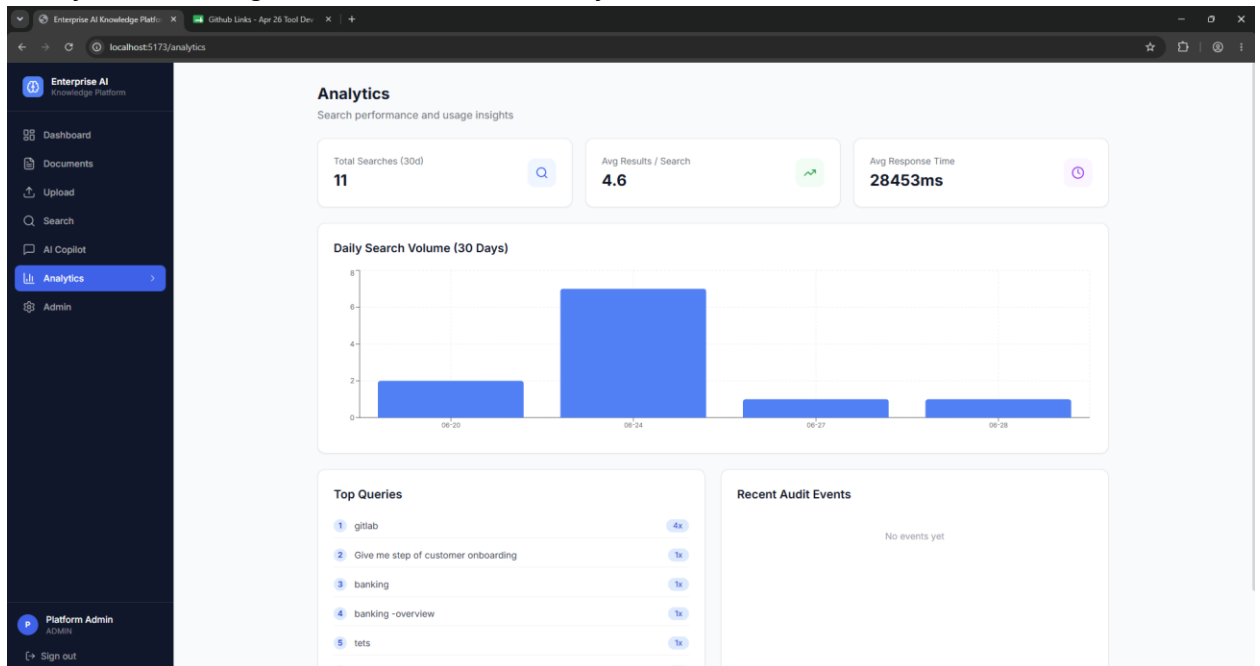






AI Copilot - Results above

Analytics – having details of search history



Admin tab :

Clicks on Add user – has the ability to add user with the permission

